

Jungle Jumper

Introduction:

Jungle Jumper is a 2D up-scrolling platformer game built in C++ and SFML 2.6. The player plays as a monkey and bounces upwards into the jungle using platforms, while dodging obstacles, and collecting power-ups. The game is inspired by Doodle Jump with modification to the theme, obstacles, and power-ups.

Gameplay:

The game starts with the monkey already on a starting platform and continuously bouncing. The player must use their A or D keys or the left and right arrow keys, to move the monkey and land on platforms, which can be normal, moving, and breaking platforms. As the player's score gets higher, obstacles start appearing - birds, falling rocks, and bee swarms. The player can collect bananas for ammo, springs for boost, and shields for protection, which come in handy against obstacles. Once you fall off the screen, the run ends. The player is then challenged to climb higher than their previous high score on the next run.

Classes / Relationships:

The project is organized around a small set of core classes, several of which are abstract base classes that let the rest of the game treat many different concrete objects the exact same way.

- Game - the top level class, it owns whichever GameState is currently active and drives the main loop. Its changeState() function is what actually switches between states. It calls onExit() on the state that is ending, moves to the new state, then calls onEnter() on it.

```
28
29 void Game::changeState(std::unique_ptr<GameState> newState) {
30     if (currentState) {
31         currentState->onExit();    //close out the old state
32     }
33     currentState = std::move(newState); // swap in the new
34     currentState->onEnter();    //set up the new state
35 }
36
```

Game.cpp - changeState() is the one function that moves the game between MainMenuState and PlayState

- GameState (abstract) - It is an interface implemented by MainMenuState and PlayState. It declares the functions every state must provide: `handleEvent()`, `update()`, `render()`, `onEnter()`, and `onExit()`. The Game never needs to know which one is active, it just calls the same functions on whichever concrete state it's holding.
- PlayState - The class that actually drives gameplay. PlayState is responsible for all that's inside the game. It owns Player, the lists of platforms, obstacles, power-ups, the bulletpool, and is responsible for spawning, collision check, scoring, and the camera each frame.
- Obstacle (abstract) - It is implemented by Bird, Bee, BeeNest and FallingRock.
- PowerUp (abstract) - It is implemented by Banana, Spring and Shield.
- ResourceManager - It is a single shared cache that loads and hands out textures.
- ObstacleFactory - A dedicated class whose only job is building the four obstacle types.
- BulletPool - It owns and recycles a fixed set of 20 bullet objects.

The abstract classes are important as they let PlayState hold one single list per category and call the same functions on every element in it, without ever needing to check what the concrete type actually is. For instance, PlayState keeps one `std::vector<std::unique_ptr<Obstacle>>` that holds Birds, Bees, FallingRocks, and BeeNests all together and every one of them responds correctly when the same function is called on it.

```
class Obstacle {
public:
    Obstacle(sf::Vector2f position, sf::Vector2f size, sf::Color color, std::unique_ptr<MovementStrategy> strategy,
        const std::string& texturePath = "");
    virtual ~Obstacle() = default;

    void update(float deltaTime); //delegates to movement strategy
    virtual void onUpdate(float deltaTime) {
    }

    //player touch obstacle from below
    virtual bool onHitFromBelow(Player& player) = 0;

    //player fell and landed cleanly top of obstacle
    virtual bool onLandedOnTop(Player& player) = 0;

    //whether this obstacle can be destroyed by bullet
    virtual bool isShootable() const {
        return true;
    }

    //bounces
    virtual bool bouncesBullets() const {
        return false;
    }

    sf::FloatRect getBounds() const;
    void draw (sf::RenderWindow& window) const;
```

Obstacle.hpp - the abstract base class that every obstacle type builds on. PlayState only ever calls `onHitFromBelow`, `onLandedOnTop`, and `isShootable` functions regardless of which concrete obstacle it's holding.

Inheritance / Composition:

The project uses both inheritance and composition for their own different reasons. Inheritance is used where an object's type is fixed for its own lifetime and different types need to respond differently to the same event. Bird, Bee, FallingRock, and BeeNest all inherit from Obstacle, and each overrides onHitFromBelow(), and onLandedOnTop() to define its own reaction. Banana, Spring, and Shield inherit from PowerUp the same way, each overriding apply() with its own effect. MainMenuState and PlayState both inherit from GameState.

Composition is used where behavior itself is more interchangeable rather than fixed. Instead of an obstacle knowing how to move itself, it holds a separate MovementStrategy object and calls move() on it. Obstacle has-a MovementStrategy, while the three concrete movements (StaticMovement, FallMovement, and AngledSwarmMovement) each inherit from MovementStrategy. So composition connects Obstacle down to MovementStrategy while inheritance connects each concrete movement up to the same class. Platforms follow the same structure. Platform has-a PlatformBehavior and NormalBehavior, MovingBehavior, and BreakingBehavior each inherit from it. This is why two unrelated classes like Bird and BeeNest can share the exact same StaticMovement object without being related to each other at all.

```
private:
    std::unique_ptr<MovementStrategy> movementStrategy;
};

//moves obstacle straight down at fixed speed but own random speed and set once spawn time
class FallMovement : public MovementStrategy {
public:
    explicit FallMovement(float speed) : speed(speed) {
    }

    void move(sf::RectangleShape& shape, float deltaTime) override {
        shape.move(0.f, speed * deltaTime);
    }

private:
    float speed;
};
```

Obstacle.hpp/FallMovement.hpp - composition connects to MovementStrategy; inheritance connects FallMovement.

Design Patterns:

Four design patterns were used in this project: Object Pool, Singleton, Factory, and Strategy.

1. Object pool - BulletPool

The object pool was used for bullets. When the player collects a banana, it gives them five bullets to shoot. Without pooling, every shot would create a brand-new bullet object and every bullet would be destroyed again once it scrolled off-screen, which is a constant allocation and deallocation that will make the game lag. Instead, BulletPool pre-allocates a fixed set of 20 bullets one time, when it is first constructed. None of them are ever created or destroyed again after that and they only switch between active and inactive.

```
class BulletPool {
public:
    explicit BulletPool(int poolSize = 20);

    //activate one currently-inactive bullet, if any are available, doesnt do anything if pool is currently in use
    void fire(sf::Vector2f position);

    //moves all active bullet and deactivate any that have scrolled above the top of visible camera view
    void update(float deltaTime, float topOfViewY);

    void draw(sf::RenderWindow& window) const;

    //exposed so playstate can check bullet vs. obstacle collisions
    std::vector<Bullet>& getBullets();

private:
    std::vector<Bullet> bullets; //fixed size after construction
};
```

BulletPool.hpp - fire() scans for inactive bullet and reuses it; if all 20 are in use, it simply does nothing

2. Singleton - ResourceManager

Singleton was used for ResourceManager, which loads and stores textures. Many different objects in the game - platforms, obstacles, powerups and more - need access to the same images. If every object loaded its own copy independently, it would waste both load time and memory. Singleton guarantees there is exactly one ResourceManager, reachable the same way from anywhere in the code, holding one shared texture cache copy. Text is not included in this cache as fonts are a different format (sf::Font instead of sf::Texture) and are loaded separately, inside MainMenuState and PlayState.

```

class ResourceManager {
public:
    //delete copy/move
    ResourceManager(const ResourceManager&) = delete;
    ResourceManager& operator=(const ResourceManager&) = delete;

    static ResourceManager& getInstance() {
        static ResourceManager instance; //created once on first use
        return instance;
    }

    sf::Texture& getTexture(const std::string& path) {
        auto it = textureCache.find(path);
        if (it != textureCache.end()) {
            return it->second; //
        }

        sf::Texture texture;
        if (!texture.loadFromFile(path)) {
            std::cerr << "ResourceManager: failed to load " << path << std::endl;
        }
        auto result = textureCache.emplace(path, std::move(texture));
        return result.first->second;
    }

private:
    ResourceManager() = default; //private constructor, only get instance() can create
    std::unordered_map<std::string, sf::Texture> textureCache;
};

```

ResourceManager.hpp - getInstance() returns the same object every time it's called, from anywhere in the codebase.

3. Factory - ObstacleFactory

The factory was used to build obstacles. There are four obstacle types, and each one needs different information to construct: a Bird only needs a position, while a Bee also needs an angle and speed. Instead of PlayState remembering every constructor's exact layout, ObstacleFactory provides one create function per type, returning the same base type, `std::unique_ptr<Obstacle>`. This lets PlayState build any obstacle without knowing how it is actually put together and lets every obstacle type live in one shared list.

```

class ObstacleFactory {
public:
    static std::unique_ptr<Obstacle> createBird(sf::Vector2f position);
    static std::unique_ptr<Obstacle> createFallingRock(sf::Vector2f position, float fallSpeed);
    static std::unique_ptr<Obstacle> createBee(sf::Vector2f position, float angleRadians, float speed);
    static std::unique_ptr<Obstacle> createBeeNest(sf::Vector2f position);
};

```

ObstacleFactory.hpp - Every function hands back the same base type, so PlayState never needs to know the concrete obstacle it created.

4. Strategy

Strategy was used twice in the game, for two separate objects, and the relationship works the same way in both classes. The object that uses the behavior - Obstacles and Platform - holds the abstract strategy through composition (has-a), while each concrete behavior reaches the same abstract class through inheritance (is-a). For obstacles, MovementStrategy is a one-method interface (move()). Each obstacle is handed one concrete movement (StaticMovement, FallMovement, or AngledSwarmMovement) once, in its own constructor and that never changes. The benefit of this method is that it can be reused. For instance, Bird and BeeNests are unrelated obstacles, but both share the exact same StaticMovement object instead of duplicating logic separately.

For platforms, it is the same idea as obstacles. Rather than growing large if/else branches to decide how each platform should act, Platform simply has-a PlatformBehavior, and one of three concrete behaviors (NormalBehavior, MovingBehavior, or BreakingBehavior) is chosen randomly whenever a platform spawns. It might not look like Strategy is really being used, since neither strategy is ever actually swapped out for a different one after an object is created. Each obstacle or platform picks its strategy once and keeps it at construction. But interchangeable does not mean a swap actually happens, it means a swap safely could happen because Obstacle only ever calls whatever strategy it is holding through the abstract interface, without ever checking which concrete one. That is what makes StaticMovement, FallMovement, and AngledSwarmMovement interchangeable, even though none of them are ever swapped while the game is running. Since nothing about the design assumes a fixed strategy, a future feature like an obstacle that switches movement once the player gets close could be added just by giving Obstacle a way to replace its strategy while running.

```
class StaticMovement : public MovementStrategy {
public:
    void move(sf::RectangleShape& shape, float deltaTime) override {
        //intentionally empty
    }
};
```

obstacles/StaticMovement.hpp - Example from the three movements that implement the same move() signature, which is what makes any of them safe to hand to any obstacle.

Open/Closed Principle and Extensibility

The Open/Closed Principle states that a class should be open for extension but closed for modification. With Jungle Jumper it should be possible to add new behavior without editing code that already works. This project follows that principle in some places, only partially in others, and not at all in one spot. It is fully followed for obstacle movement, since `Obstacle::update()` only calls `movementStrategy->move()`, so a new movement type means writing one new class and handing it to an obstacle, without needing to open `Obstacle.cpp`. Creating new obstacle types needs one exception. `PlayState` never changes when a new obstacle is added, since it only calls `ObstacleFactory::createX`, though `ObstacleFactory` itself needs one new function each time. It is not followed at all for power-ups, since `Banana`, `Spring`, and `Shield` are built directly inside `PlayState.cpp` rather than through a factory, so adding one means editing existing code directly. It does not apply to changing an existing power-up's effect, since there are no swappable "effect" objects the way there is a swappable `MovementStrategy`.

```
void Obstacle::update(float deltaTime) {  
    movementStrategy->move(shape, deltaTime);    //never edit to add new movement  
    onUpdate(deltaTime); //empty by default  
}
```

`Obstacle.cpp` - never edited to add new movement types or obstacle-specific behavior, `onUpdate()` is a hook subclass like bee nest override.

Due to this, along with Strategy and Factory, parts of this project can act as a template for another developer to extend. A new movement, platform behavior, or obstacle type can all be added by implementing the right interface without touching the rest of the codebase. Extending only applies to a few specific parts of the game since `PlayState` is one large file that handles a lot at once, and there is no way to add features without editing that code directly.

Lessons Learned

One of the biggest challenges in making this project was keeping track of how everything fit together as the number of classes grew. Obstacle, PowerUp, MovementStrategy, and more all interact in different ways and it took a while to build a clear model of which class owns what and which relationships were inheritance and composition. As I got closer to finishing the game, I

began to understand it more clearly. In addition to that, PlayState kept growing - almost every new feature, whether a new obstacle, new power-up, or camera behavior, ended up adding more to it, and it became the largest class, which made it harder to navigate over time.

One example of a design assumption I had that turned out to be wrong was the BeeNest. It originally released its bees the moment it was created, which happened off-screen, above the camera, before the player could see it. This meant the bees would sometimes trigger before the player arrived or never trigger at all if the player climbed past quickly. The fix required redoing the trigger condition from “when this was created” to “is the player actually near.” It taught me that sometimes the problem is not the code itself, but the assumption behind how it was designed.

One insight that stood out while building is that I misunderstood what “interchangeable” actually means in the Strategy pattern. I thought that being interchangeable meant a strategy was actually being swapped out during runtime. In reality, neither MovementStrategy nor PlatformBehavior ever does this. They pick strategy once, at construction. The real meaning turned out to be about what the design allows, not what it actively does. Since Obstacle only ever calls its strategy through abstract interface without checking which concrete one it has, any movement type could be handed safely to any obstacle without breaking anything, and that safety is what interchangeable really means.

What Would Be Done Differently

Something that I would have done differently is to implement the design patterns earlier in the process, rather than writing the long code first and refactoring afterward. Singleton and Factory are good examples of this - both were planned early on, but ResourceManager and ObstacleFactory were not actually refactored into their pattern form until the end of the project, which meant I was dealing with the problems those patterns were meant to solve for most of development. Refactoring afterward did turn out to be a good way to actually understand how each pattern helped, but planning the architecture around them from the start would likely have avoided some of that rework, especially around PlayState, which kept growing as more features were added.

Another approach would have been to give power-ups to their own factory, similar to ObstacleFactory. Right now, Banana, Spring, and Shield are constructed directly with

make_unique inside PlayState, in two separate places, instead of through a dedicated factory. This is different from how obstacles are handled and centralizing it would remove the duplication between those two places.

I also considered a few gameplay additions. One is more obstacle and platform variety, specifically a new power-up - a spring - that throws the player sideways instead of straight up, and separately, a new kind of platform that behaves like a normal platform but shoves the player left or right. These will add a different kind of challenge. The other is music. I attempted it but my VirtualBox audio issue on the development machine prevented it from working correctly. I would also like to add a boss encounter around a score of 20,000, along with additional obstacles that unlock once the player passes a score of 10,000.